

Working on IAM

Working on IAM (Identity and Access Management) involves creating and managing users, roles, and permissions to securely control access to AWS services and resources. The main purpose of IAM is to

ensure that only authorized users can access specific resources based on their roles and responsibilities.

It allows administrators to assign policies, manage credentials, and implement security practices such

as least privilege, where users are given only the permissions they need. IAM also helps avoid the use

of the root account for regular tasks, improving overall account security.

By using IAM effectively, organizations can protect their data, reduce security risks, and maintain proper control over who can access and modify AWS resources.

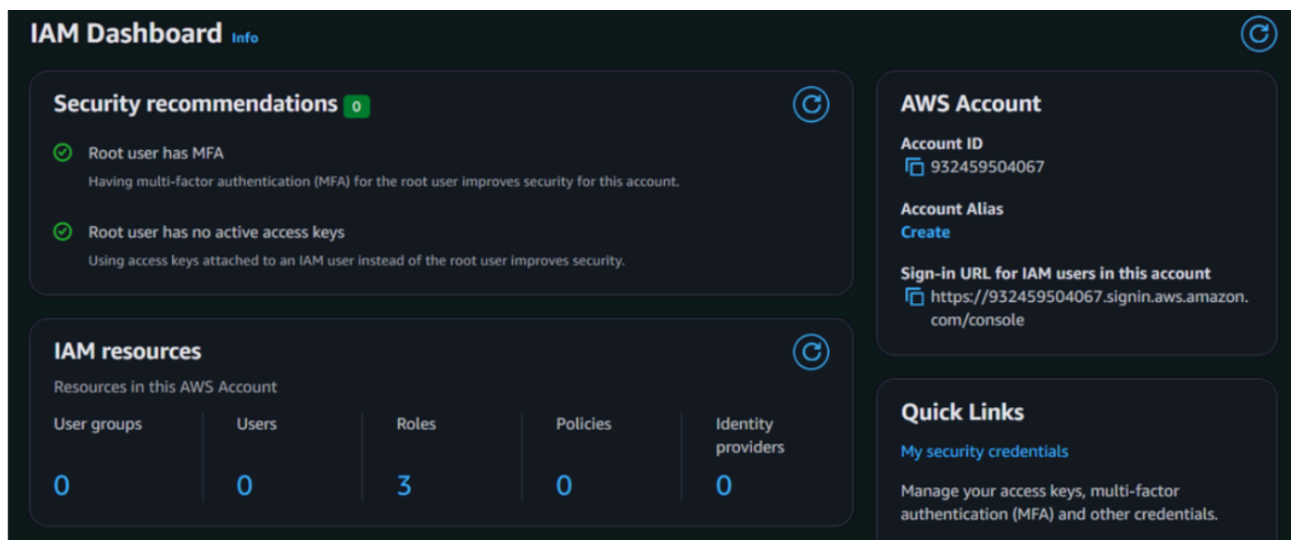
Step 1 : IAM Dashboard

The IAM Dashboard is the main interface for managing access and security in AWS. It allows users to

create and manage IAM users, roles, and permissions. From this dashboard, you can view security

status, apply policies, and control who has access to AWS resources. It also provides important recommendations like enabling MFA for better account protection.

Working on IAM

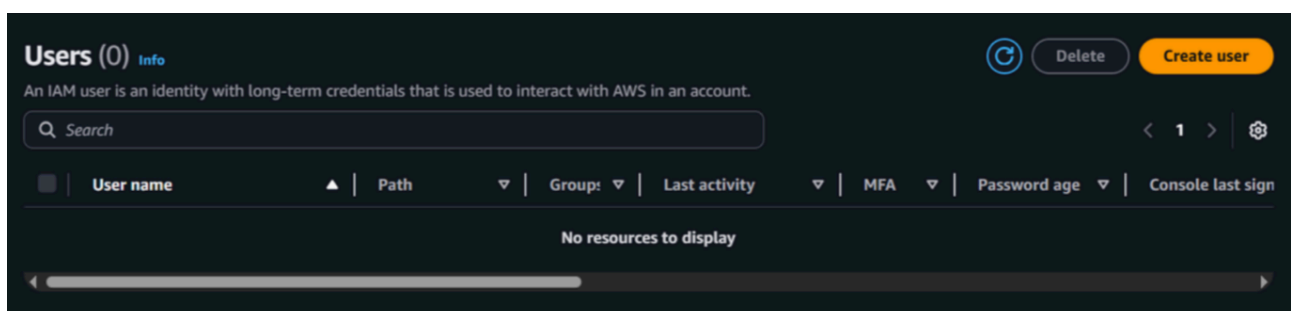


Step 2 : Creating a User

This step involves creating a new IAM user to provide access to AWS resources. A unique username is

assigned, and the type of access (programmatic or console) is selected. It ensures that individuals can

securely access the AWS account with specific credentials instead of using the root account. Step



Step 3 : Naming the User and Configuring Settings

In this step, a username is assigned and access settings are configured. The Provide user access option is enabled to allow the user to log in to AWS. Under permissions, the Attach policies directly option is selected to assign specific policies to the user, defining what actions they can perform on AWS resources.

Set permissions
Add user to an existing group or create a new one. Using groups is a best-practice way to manage user's permissions by job functions. [Learn more](#)

Permissions options

- ☒ **Add user to group**
Add user to an existing group, or create a new group. We recommend using groups to manage user permissions by job function.
- ☐ **Copy permissions**
Copy all group memberships, attached managed policies, and inline policies from an existing user.
- ☐ **Attach policies directly**
Attach a managed policy directly to a user. As a best practice, we recommend attaching policies to a group instead. Then, add the user to the appropriate group.

Get started with groups
Create a group and select policies to attach to the group. We recommend using groups to manage user permissions by job function, AWS service access, or custom permissions. [Learn more](#)

Create group

Set permissions boundary - optional

Cancel Previous Next

Set permissions
Add user to an existing group or create a new one. Using groups is a best-practice way to manage user's permissions by job functions. [Learn more](#)

Permissions options

- ☒ **Add user to group**
Add user to an existing group, or create a new group. We recommend using groups to manage user permissions by job function.
- ☐ **Copy permissions**
Copy all group memberships, attached managed policies, and inline policies from an existing user.
- ☐ **Attach policies directly**
Attach a managed policy directly to a user. As a best practice, we recommend attaching policies to a group instead. Then, add the user to the appropriate group.

Get started with groups
Create a group and select policies to attach to the group. We recommend using groups to manage user permissions by job function, AWS service access, or custom permissions. [Learn more](#)

Create group

Set permissions boundary - optional

Cancel Previous Next

Set permissions
Add user to an existing group or create a new one. Using groups is a best-practice way to manage user's permissions by job functions. [Learn more](#)

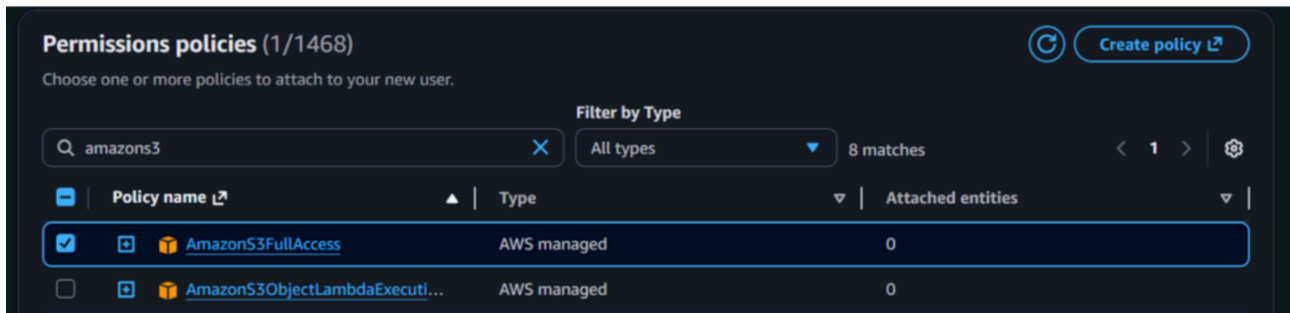
Permissions options

- ☐ **Add user to group**
Add user to an existing group, or create a new group. We recommend using groups to manage user permissions by job function.
- ☐ **Copy permissions**
Copy all group memberships, attached managed policies, and inline policies from an existing user.
- ☒ **Attach policies directly**
Attach a managed policy directly to a user. As a best practice, we recommend attaching policies to a group instead. Then, add the user to the appropriate group.

Step 4 : Selecting Permission Policy

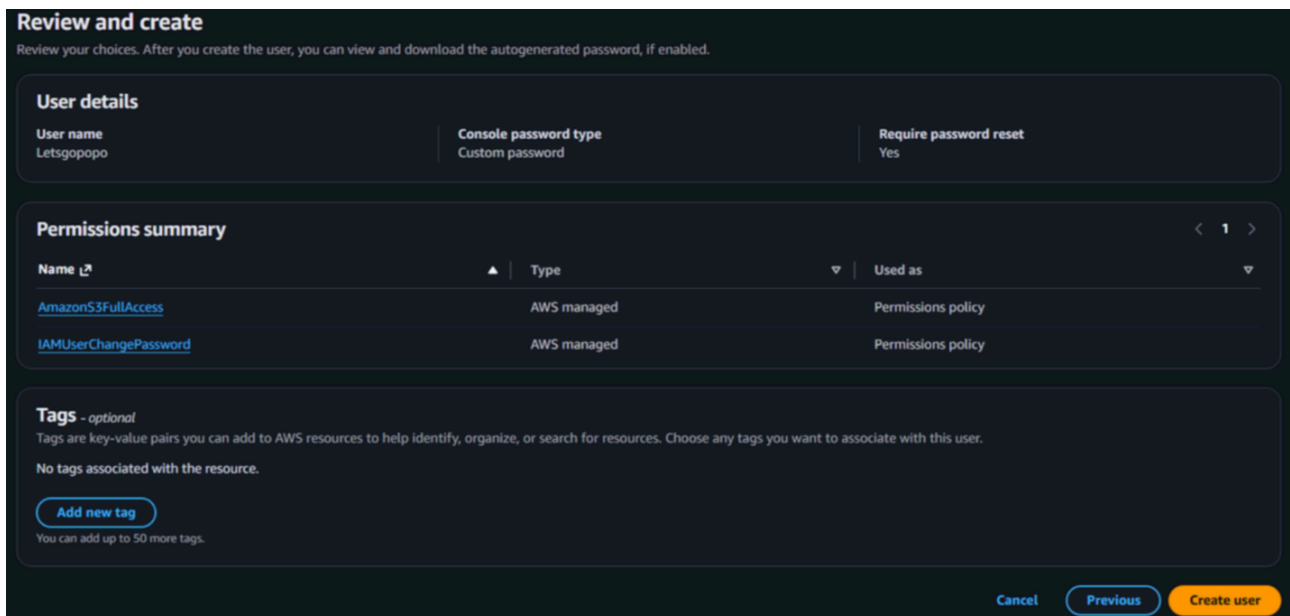
In this step, the AmazonS3FullAccess policy is selected from the permissions list. This policy grants the user full access to Amazon S3 services, allowing them to create, view, edit, and delete S3

resources. Assigning this policy ensures the user has complete control over S3 operations within the AWS account.



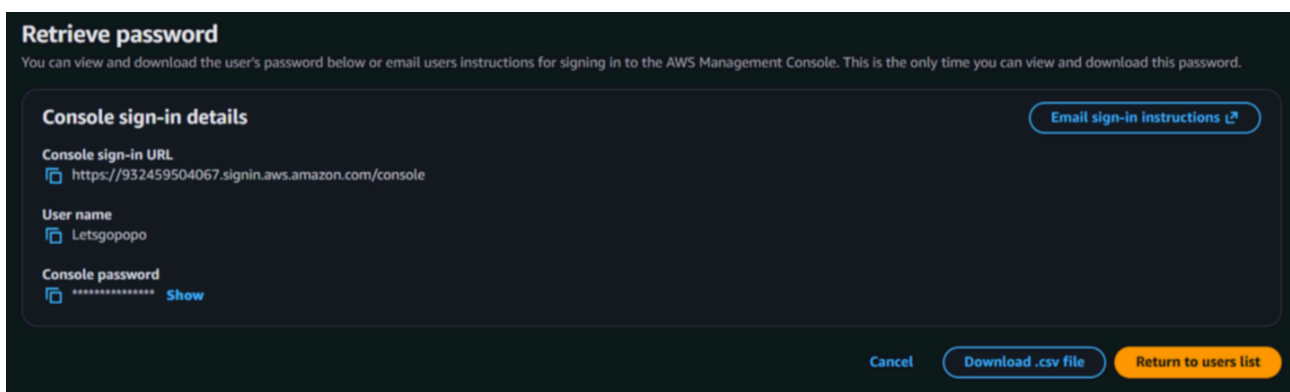
Step 5 : Review and Create User

In this step, all the configured details are reviewed before creating the user. This includes the username, selected permissions (policies), and any tags added. The permissions summary provides a clear overview of access rights assigned to the user. After verifying the details, the user is created successfully.



Step 6 : Retrieving Password and Sign-In Details

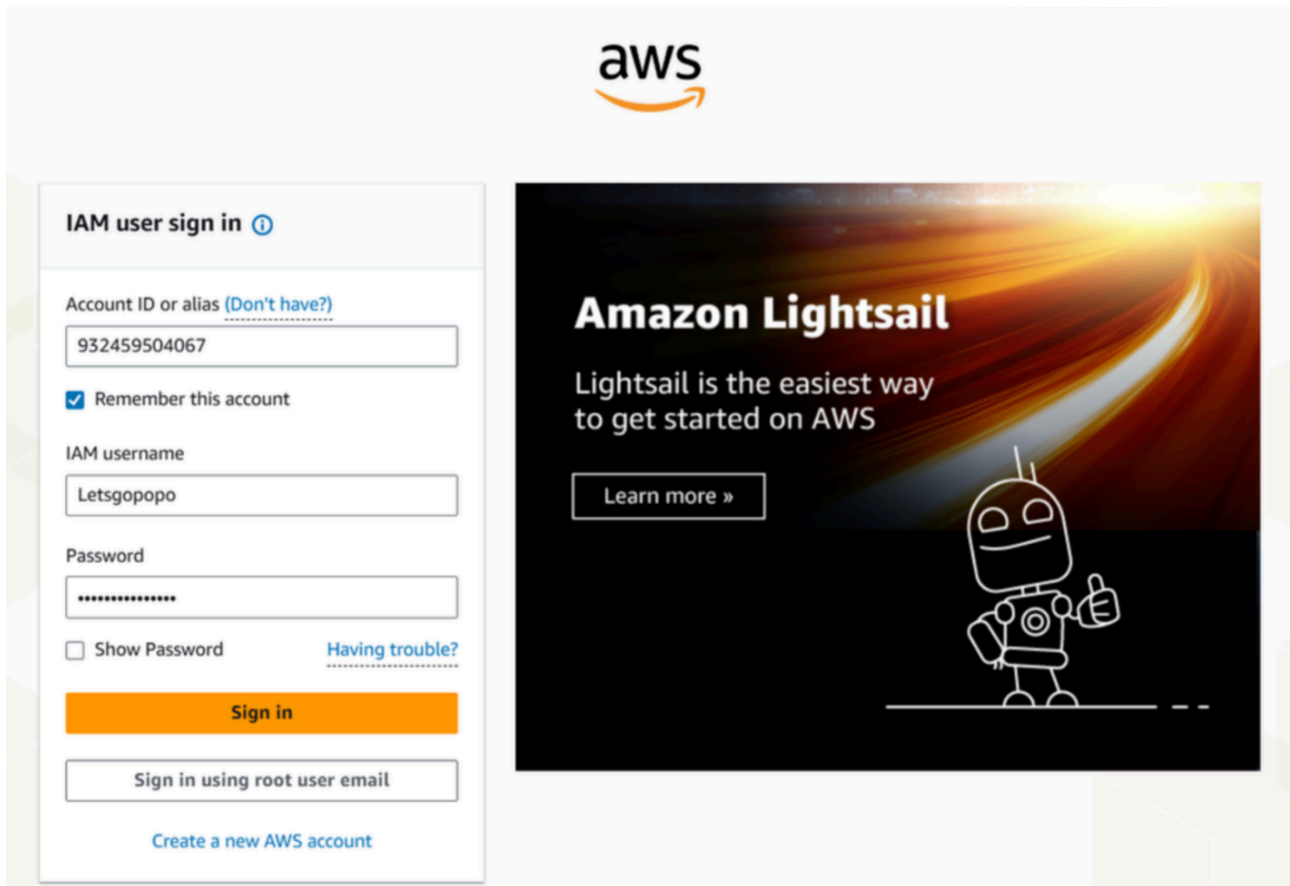
In this step, the login credentials for the IAM user are displayed. This includes the console sign-in URL, username, and password. These details are used by the user to access the AWS Management Console. It is important to store this information securely or download it for future use.



Step 7 : Sharing Sign-In Details

The sign-in URL is shared with the user. The user is instructed to open the link in an incognito window

or a different browser and log in using the provided username and password. This ensures secure access and prevents conflicts with existing AWS sessions.



Step 8 : AWS Console Home (Incognito Mode)

In this step, the IAM user successfully logs into the AWS Management Console using the provided credentials in an incognito window. The Console Home page is displayed, showing access to various AWS services. This confirms that the user account has been created correctly and is able to access the assigned resources.

